

MICROSERVICES COURSE

Spring Boot & Spring Cloud — Professional Edition

SESSION 18

CQRS Pattern

Command Query Responsibility Segregation • Read Models • Spring Data Projections • Domain Events

Complete Student Tutorial — Enterprise E-Commerce Platform

1. Session Information

Item	Details
Session	
	18
Title	
	CQRS Pattern
Topics	
	CQRS, commands, queries, read models, Spring Data projections, application events
Course context	
	Enterprise E-Commerce Platform continuous capstone
Starting point	
	product-service with JPA and the Redis cache introduced earlier in the course
Today's scope	
	Logical CQRS using the same database
Out of scope	
	Separate read database, distributed synchronization, Event Sourcing implementation, Axon Framework

2. Learning Objectives

- Recognize when one entity/repository pair is being forced to support conflicting write and read responsibilities.
- Define CQRS and distinguish Commands from Queries.
- Separate Product writes into ProductCommandService and reads into ProductQueryService.
- Build read-optimized models with Spring Data projections and DTOs.
- Use joins, flattened fields and display-oriented data without polluting the write entity.
- Publish product-change events and react safely after transaction commit.
- Connect product changes to Redis cache invalidation.
- Test command and query responsibilities independently.
- Choose between no separation, projections only, logical CQRS and full CQRS.
- Explain why CQRS and Event Sourcing are different patterns.

3. Agenda

Part	Topic
1	
	The problem: one model, two conflicting jobs

2	
	CQRS fundamentals
3	
	Engineering judgement: when NOT to use CQRS
4	
	Command side
5	
	Read models and Spring Data projections
6	
	Query side and REST APIs
7	
	Events and transaction boundaries
8	
	CQRS + Redis
9	
	Testing
10	
	Lab, quiz, homework and summary

PART 1 — THE PROBLEM

4. One Model, Two Conflicting Jobs

product-service has grown. It now supports normal product writes and simple product lookups, but operations also request a dashboard showing each product, category name, stock information and order-related statistics. The dashboard does not want the Product entity exactly as the domain writes it.

- Product ID and name
- Current price
- Category name
- Current stock
- Order count for a reporting period
- Display-oriented calculated fields

The common mistake is to place all dashboard logic inside the same `ProductService.findAll()` method used by simple application flows. That creates an overloaded abstraction: domain writes, simple lookups, joins, aggregations and presentation requirements all change together.

Write-side concern	Read-side concern
Validate invariants	
	Return exactly what the consumer needs
Represent domain state correctly	
	Shape data for UI/reporting
Usually normalized	
	Often denormalized
Changes with business rules	
	Changes with screen/report requirements
Correctness first	
	Read efficiency and usability first

The key problem is not only performance. The same model now has two very different reasons to change.

5. The Product-Service Story

Suppose a developer adds category joins, Inventory Service calls and weekly order counts directly to `ProductService.findAll()`. A latency-sensitive path that needs a simple product lookup now inherits dashboard complexity. Next month, another dashboard field is requested, so the same method grows again.

CQRS begins by asking a simple question: should the same model and responsibility really serve both of these jobs?

PART 2 — CQRS FUNDAMENTALS

6. What Is CQRS?

CQRS means Command Query Responsibility Segregation.

Command	Query
Changes state	
	Reads state
Expresses intent	
	Answers a question
Enforces business rules	
	Does not perform business mutations
Usually transactional write	
	Usually read-only transaction
Returns minimal acknowledgement, ID or command response	
	Returns data for the caller
Create, update, delete	
	Find, list, search, report

Student rule: COMMAND = 'Please change something.' QUERY = 'Please tell me something.'

CQRS does NOT automatically require two databases, Kafka, Event Sourcing or Axon. This session uses lightweight logical CQRS inside one microservice and one database.

7. Request Flow

POST / PUT / DELETE

Client → Command Controller → `ProductCommandService`
→ `validate` → `repository` → `commit` → `ProductChangedEvent`

GET

Client → Query Controller → `ProductQueryService`
→ `projection/read model` → `database or cache` → `response`

8. Core Rules

- A command changes state; a query does not change business state.
- Commands own business invariants.
- Queries are shaped around consumer needs.
- Read models do not have to mirror JPA entities.
- Separation can be logical first and physical later only when justified.

PART 3 — ENGINEERING JUDGEMENT

9. Full CQRS vs Projection vs No Separation

Situation	Decision
Simple CRUD; read and write shapes nearly identical	
	No separation
A read needs selected fields, joins or a different shape	
	Projection/DTO
Write invariants and read requirements are pulling the model in different directions	
	Separate Command and Query services
Read/write scale independently or need different storage	
	Full CQRS with separate read store
State history must be the durable source of truth	
	Consider Event Sourcing; a different pattern

10. When NOT to Use CQRS

- Small, simple CRUD service with no conflicting read requirements.
- Separation is added only because the pattern sounds advanced.
- The team must manage complexity that has no real business benefit.
- A simple DTO/projection already solves the read problem.

CQRS is not a synonym for good architecture. It is a specific answer to a specific conflict. Use the smallest design that solves the real problem.

PART 4 — COMMAND SIDE

11. Suggested Package Structure

```
com.example.productservice
├── command
│   ├── ProductCommandService.java
│   └── dto
├── query
│   ├── ProductQueryService.java
│   ├── ProductSummaryProjection.java
│   └── ProductSummaryResponse.java
├── event
│   └── ProductChangedEvent.java
├── controller
├── domain
└── repository
```

12. Command Request DTOs

```
public record CreateProductRequest(
    String name,
    String description,
    BigDecimal price,
    Long categoryId
) {}

public record UpdateProductRequest(
    String name,
    String description,
    BigDecimal price,
```

```
        Long categoryId
    ) {}
}
```

13. ProductChangedEvent

```
public record ProductChangedEvent(
    Long productId,
    ChangeType changeType
) {
    public enum ChangeType {
        CREATED, UPDATED, DELETED
    }
}
```

14. ProductCommandService

The command side owns state changes. It validates input, loads existing state when required, enforces invariants, persists the change and announces that a change occurred.

```
@Service
@RequiredArgsConstructor
public class ProductCommandService {

    private final ProductRepository productRepository;
    private final ApplicationEventPublisher eventPublisher;

    @Transactional
    public Long create(CreateProductRequest request) {
        validatePrice(request.price());

        Product product = new Product();
        product.setName(request.name());
        product.setDescription(request.description());
        product.setPrice(request.price());
        product.setCategoryId(request.categoryId());

        Product saved = productRepository.save(product);

        eventPublisher.publishEvent(
            new ProductChangedEvent(saved.getId(),
                ProductChangedEvent.ChangeType.CREATED));

        return saved.getId();
    }

    @Transactional
    public void update(Long id, UpdateProductRequest request) {
        Product product = productRepository.findById(id)
            .orElseThrow(() -> new ProductNotFoundException(id));

        validatePrice(request.price());

        product.setName(request.name());
        product.setDescription(request.description());
        product.setPrice(request.price());
        product.setCategoryId(request.categoryId());

        eventPublisher.publishEvent(
            new ProductChangedEvent(id,
                ProductChangedEvent.ChangeType.UPDATED));
    }

    @Transactional
    public void deleteById(Long id) {
        if (!productRepository.existsById(id)) {
```

```

        throw new ProductNotFoundException(id);
    }

    productRepository.deleteById(id);

    eventPublisher.publishEvent(
        new ProductChangedEvent(id,
            ProductChangedEvent.ChangeType.DELETED));
    }

    private void validatePrice(BigDecimal price) {
        if (price == null ||
            price.compareTo(BigDecimal.ZERO) <= 0) {
            throw new InvalidProductException(
                "Price must be positive");
        }
    }
}

```

Why load the entity before update? The command is changing an existing aggregate. Loading the current state creates a clear place to enforce rules and avoids blindly saving an external detached entity.

15. Command REST Controller

```

@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductCommandController {

    private final ProductCommandService commandService;

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Map<String, Long> create(
        @RequestBody CreateProductRequest request) {
        return Map.of("id", commandService.create(request));
    }

    @PutMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void update(@PathVariable Long id,
        @RequestBody UpdateProductRequest request) {
        commandService.update(id, request);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) {
        commandService.deleteById(id);
    }
}

```

PART 5 — READ MODELS AND PROJECTIONS

16. What Is a Read Model?

A read model is designed for the consumer. It may contain only selected fields, flattened relationships, joins or calculated values. It does not need to be the same shape as the Product entity.

17. Interface-Based Projection

```

public interface ProductSummaryProjection {

```

```

    Long getId();
    String getName();
    BigDecimal getPrice();
    String getCategoryName();

    default String getDisplayLabel() {
        return getName() + " (" + getCategoryName() + ")";
    }
}

```

Spring Data maps query aliases to projection getters. The alias `categoryName` therefore corresponds to `getCategoryName()`.

18. Projection Repository Queries

```

public interface ProductRepository
    extends JpaRepository<Product, Long> {

    @Query("""
        SELECT p.id AS id,
               p.name AS name,
               p.price AS price,
               c.name AS categoryName
        FROM Product p
        JOIN Category c ON p.categoryId = c.id
        WHERE p.id = :id
        """)
    Optional<ProductSummaryProjection>
        findSummaryById(@Param("id") Long id);

    @Query("""
        SELECT p.id AS id,
               p.name AS name,
               p.price AS price,
               c.name AS categoryName
        FROM Product p
        JOIN Category c ON p.categoryId = c.id
        """)
    List<ProductSummaryProjection> findAllSummaries();
}

```

19. DTO-Based Projection Alternative

```

public record ProductSummaryResponse(
    Long id,
    String name,
    BigDecimal price,
    String categoryName
) {
    public String displayLabel() {
        return name + " (" + categoryName + ")";
    }
}

```

Interface projection	DTO/record projection
Minimal mapping code	
	Explicit response construction
Getter aliases are important	
	Constructor fields are explicit
Convenient for simple repository reads	
	Useful for deliberate API/read structures

PART 6 — QUERY SIDE

20. ProductQueryService

```
@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class ProductQueryService {

    private final ProductRepository productRepository;

    public ProductSummaryProjection findById(Long id) {
        return productRepository.findSummaryById(id)
            .orElseThrow(() -> new ProductNotFoundException(id));
    }

    public List<ProductSummaryProjection> findAll() {
        return productRepository.findAllSummaries();
    }
}
```

`readOnly = true` expresses query intent and can enable persistence-provider read optimizations. It is useful, but it is not a substitute for designing a method as a genuine query.

21. Query REST Controller

```
@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductQueryController {

    private final ProductQueryService queryService;

    @GetMapping("/{id}")
    public ProductSummaryProjection findById(
        @PathVariable Long id) {
        return queryService.findById(id);
    }

    @GetMapping
    public List<ProductSummaryProjection> findAll() {
        return queryService.findAll();
    }
}
```

CQRS does not require different URLs. It requires separation of responsibilities. Keeping GET and POST/PUT/DELETE under the same REST resource path is perfectly valid.

PART 7 — EVENTS AND TRANSACTIONS

22. Why an Event When Both Sides Share the Same Database?

The event is not needed to copy data to the read model in today's design. The projection reads the same database. The event provides a decoupling seam: the command side announces a change without knowing which concerns react.

- Evict a Redis cache.
- Refresh a future materialized read model.
- Update a search index.
- Record an application metric.
- Trigger future integration through a separate reliable mechanism.

23. Transaction-Safe Cache Eviction

Important: a normal event listener may execute before the surrounding transaction has successfully committed. If the transaction rolls back, a listener may already have performed cache or external work. For post-commit cache invalidation, use `TransactionalEventListener` with `AFTER_COMMIT`.

```
@Component
@RequiredArgsConstructor
@Slf4j
public class ProductCacheEvictionListener {

    private final CacheManager cacheManager;

    @TransactionalEventListener(
        phase = TransactionPhase.AFTER_COMMIT)
    public void onProductChanged(
        ProductChangedEvent event) {

        Cache products = Objects.requireNonNull(
            cacheManager.getCache("products"));

        products.evict(event.productId());
        products.evict("all");

        log.info(
            "[CQRS] Cache evicted for product {} due to {}",
            event.productId(), event.changeType());
    }
}
```

24. Synchronous vs Asynchronous Listeners

Synchronous default	Asynchronous
Simple in-process work	
	Runs through another thread/executor
Easy failure reasoning	
	Needs executor/error-handling design
Good for small local actions	
	Useful for slow non-critical work
Not durable messaging	
	Still not durable messaging

Do not add `@Async` simply because an event exists. Asynchronous execution and reliable distributed delivery are different concerns.

PART 8 — CQRS + REDIS

25. End-to-End Cache Flow

1. GET /api/products/10 reaches the query side.
2. The read result is returned from cache if available; otherwise the projection reads the database.
3. A create, update or delete reaches `ProductCommandService`.
4. The command validates and writes state.
5. The transaction commits successfully.
6. `ProductChangedEvent` is handled `AFTER_COMMIT`.
7. The listener evicts stale product and list cache keys.
8. The next query rebuilds or retrieves the fresh read result.

This keeps cache concerns out of ProductCommandService. The command service knows about the domain change, not the cache implementation.

PART 9 — TESTING

26. Command-Side Tests

- Positive price creates a product.
- Zero/negative price throws InvalidProductException.
- Missing product update fails.
- Delete existing product succeeds.
- Correct change event is published.

```
@ExtendWith(MockitoExtension.class)
class ProductCommandServiceTest {

    @Mock ProductRepository productRepository;
    @Mock ApplicationEventPublisher eventPublisher;

    @InjectMocks ProductCommandService service;

    @Test
    void create_shouldRejectNonPositivePrice() {
        var request = new CreateProductRequest(
            "Phone", "Demo", BigDecimal.ZERO, 1L);

        assertThrows(InvalidProductException.class,
            () -> service.create(request));

        verifyNoInteractions(productRepository);
    }
}
```

27. Query-Side Tests

- Projection contains expected values.
- Missing projection produces the expected exception.
- Query side never calls save/delete.
- JPQL aliases correctly map to projection getters.

28. Event and Integration Tests

9. Create a product.
10. Query it and verify the projection.
11. Update the product.
12. Verify successful commit.
13. Verify cache eviction.
14. Query again and verify the fresh result.
15. Delete the product and verify query behavior.

PART 10 — CQRS VS EVENT SOURCING

29. Do Not Confuse Them

CQRS	Event Sourcing
Separates read and write responsibilities	
	Stores state history as durable events
Does not require events	
	Uses events as the source of truth
Can use normal relational persistence	
	Requires event-history reconstruction/projections
Can use one database	
	Often needs specialized infrastructure
Can exist alone	
	Often pairs well with CQRS but is independent

CQRS asks: Should reads and writes use different responsibilities/models? Event Sourcing asks: How is domain state and its history stored? They can be combined, but neither automatically requires the other.

30. Common Mistakes and Troubleshooting

Problem	Better approach
Separate databases immediately	
	Start with logical CQRS if it solves the problem
Return JPA entities everywhere	
	Use minimal command responses and read models
Validate business rules in queries	
	Keep invariants on the command side
Add reporting fields to Product entity	
	Use a projection/DTO
Evict cache directly from every command	
	Use a decoupled post-commit listener
Use in-process events as durable messaging	
	Use a proper broker/outbox design when reliability is required
Assume @Async makes delivery reliable	
	Async is not durability
Projection field is null	
	Check query alias and getter name
JPQL fails	
	Use entity and Java field names, not table/column names

31. Live Coding Checklist

16. Create command/query packages.
17. Create request DTOs.
18. Create ProductChangedEvent.
19. Move create/update/delete to ProductCommandService.
20. Keep validation on the command side.
21. Create ProductSummaryProjection.
22. Add projection repository queries.
23. Create ProductQueryService.
24. Create command/query controllers.
25. Add AFTER_COMMIT cache listener.
26. Run unit tests.

27. Demonstrate create → query → update → cache eviction → query.

32. Hands-On Lab

Goal: Refactor product-service into lightweight CQRS without introducing a second database.

28. Create ProductCommandService for all writes.
29. Create ProductQueryService for all reads.
30. Create ProductSummaryProjection with id, name, price and category name.
31. Add projection repository queries.
32. Create separate create/update request DTOs.
33. Publish ProductChangedEvent after writes.
34. Evict Redis product cache after successful commit.
35. Write command and query tests.
36. Demonstrate that a dashboard field can change without changing the write entity.

Acceptance Criteria

- No query changes product state.
- No command performs dashboard aggregation.
- Read endpoints return projections/read DTOs.
- Write endpoints validate invariants.
- Stale cache entries are invalidated after successful writes.
- Tests pass.
- Student can explain why a separate database is not required today.

33. Assessment

Area	Weight
Feature correctness	
	70%
Tests	
	20%
Code quality and CQRS separation	
	10%

34. Project Progress

- Product-service has an explicit command side.
- Product-service has a read-optimized query side.
- Redis invalidation is decoupled from command logic.
- The architecture has a seam for future read-store evolution.
- Event Sourcing remains intentionally out of scope for this session.

35. Quiz

Q: What does CQRS stand for?

Answer: Command Query Responsibility Segregation.

Q: Must CQRS use two databases?

Answer: No. Logical CQRS can use one database.

Q: What does a command do?

Answer: Express intent, validate rules and change state.

Q: What does a query do?

Answer: Return information without changing business state.

Q: Why use a projection?

Answer: To shape read data without polluting the write entity.

Q: Why use AFTER_COMMIT for cache eviction?

Answer: To avoid reacting to a change that later rolls back.

Q: Is CQRS the same as Event Sourcing?

Answer: No. They solve different problems.

Q: When should CQRS be avoided?

Answer: When the read/write conflict does not justify the added complexity.

36. Homework

37. Add one display-oriented field to the read projection.
38. Add product search by name using a read-side projection/DTO.
39. Test that invalid prices are rejected.
40. Create an integration test for create → query → update → cache eviction → query.
41. Document one justified CQRS case and one unjustified case in the capstone.

37. Final Summary

CQRS is not about making every microservice more complicated. It is about recognizing when reading and writing are asking one model to perform conflicting jobs. In this session, product-service uses lightweight CQRS: a command side for validated state changes, a query side with read-optimized projections, one relational database, and internal events for decoupled post-write actions such as cache eviction.

The key engineering lesson is: start simple, identify a real read/write conflict, apply the smallest separation that solves it, and introduce separate stores or heavier patterns only when business scale and complexity genuinely require them.

38. Session Completion Checklist

- I can distinguish a command from a query.
- I can identify read/write model conflict.
- I know when a projection alone is sufficient.
- I can implement command and query services.
- I can create interface and DTO projections.
- I understand post-commit event handling.
- I can decouple cache invalidation from command logic.
- I can explain CQRS versus Event Sourcing.
- I know CQRS is a design choice, not a default requirement.